AI+X 중급 2차 프로젝트
중간점검발표 -2

202310843 송예린
202310847 심영주
202310856 유세리

목차

1. 3번파일_모델링
2. 자율 프로젝트 주제

1. 3번파일_모델링

1. 3번파일_모델링(사용데이터)

[3-1] 학습 및 테스트 데이터 로딩

```
# train_x.csv / train_y.csv / test_x.csv / test_y.csv 4개의 데이터 로딩
train_x = pd.read_csv('data/train_x.csv')
train_y = pd.read_csv('data/train_y.csv')

test_x = pd.read_csv('data/test_x.csv')
test_y = pd.read_csv('data/test_y.csv')
```

```
train_x.describe()
```

	Unnamed: 0	SO2	CO	O3	NO2	PM10	PM25	기온(°C)	강수량(mm)	풍속(m/s)	...	시정(10
count	8431.000000	8431.000000	8431.000000	8431.000000	8431.000000	8431.000000	8431.000000	8431.00000	8431.000000	8431.000000	...	8431.0000
mean	4351.437077	0.003387	0.506903	0.029111	0.023430	39.115645	20.549638	13.58451	0.134088	2.304839	...	1715.1245
std	2506.138488	0.000827	0.210349	0.019704	0.014487	47.900614	16.583678	10.85166	1.181951	1.121013	...	524.2707
min	24.000000	0.001000	0.200000	0.000000	0.003000	3.000000	1.000000	-18.50000	0.000000	0.000000	...	33.0000
25%	2171.500000	0.003000	0.400000	0.015000	0.012000	18.000000	10.000000	5.90000	0.000000	1.500000	...	1678.0000
50%	4346.000000	0.003000	0.400000	0.028000	0.019000	30.000000	16.000000	14.30000	0.000000	2.200000	...	2000.0000
75%	6510.500000	0.004000	0.600000	0.040000	0.031000	46.000000	25.000000	22.40000	0.000000	2.900000	...	2000.0000
max	8758.000000	0.011000	2.000000	0.137000	0.082000	942.000000	154.000000	36.30000	64.700000	8.300000	...	2000.0000

8 rows × 31 columns

```
train_y.describe()
```

	PM10_1
count	8431.000000
mean	39.135334
std	47.914243
min	3.000000
25%	18.000000
50%	30.000000
75%	46.000000
max	942.000000

21년도 데이터 → train → y값 제외 → train_x

22년도 데이터 → test → y값 제외 → test_x

t+1시점의 미세먼지 농도

따로 분리 → y-> train(21년), test(22년)

1. 3번파일_모델링(전처리)

```python
# 스케일링
scaler = MinMaxScaler()
train_x_scaled = scaler.fit_transform(train_x)
test_x_scaled = scaler.transform(test_x)
```

입력 특성의 스케일 차이를 줄이기 위해 MinMaxScaler 적용
→ 학습 안전성 향상

타겟값이 정규화되지 않은 회귀 문제이기 때문에
train_y는 스케일링 X

1. 3번파일_모델링(모델링)

- RandomForest Regressor
- XGBoost Regressor
- LightGBM Regressor
- CatBoost Regressor

- MSE
- R2 score

1. Random Forest Regressor

```python
# 1. Random Forest Regressor
rf_model = RandomForestRegressor(n_estimators=100, random_state=42)
rf_model.fit(X_train, y_train)

rf_preds = rf_model.predict(X_val)
rf_mse = mse(y_val, rf_preds)
print(f'RandomForest MSE: {rf_mse}')
print(f'RandomForest R2 Score: {r2_score(y_val, rf_preds)}')
```

```
RandomForest MSE: 72.39298873740368
RandomForest R2 Score: 0.9738970758039409
```

```python
# Test 데이터로 성능을 평가해보세요.
rf_preds = rf_model.predict(test_x_scaled)
rf_mse = mse(test_y, rf_preds)
print(f'RandomForest MSE: {rf_mse}')
print(f'RandomForest R2 Score: {r2_score(test_y, rf_preds)}')
```

```
RandomForest MSE: 40.85358471679688
RandomForest R2 Score: 0.9266677940290309
```

2. XGBoost Regressor

```python
# 2. XGBoost Regressor
xgb_model = XGBRegressor(n_estimators=100, learning_rate=0.1, random_state=42)
xgb_model.fit(X_train, y_train)

xgb_preds = xgb_model.predict(X_val)
xgb_mse = mse(y_val, xgb_preds)
print(f'XGBoost MSE: {xgb_mse}')
print(f'XGBoost R2 Score: {r2_score(y_val, xgb_preds)}')
```

```
XGBoost MSE: 99.25296785731418
XGBoost R2 Score: 0.9642121047714838
```

```python
# Test 데이터로 성능을 평가해보세요.
xgb_preds = xgb_model.predict(test_x_scaled)
xgb_mse = mse(test_y, xgb_preds)
print(f'XGBoost MSE: {rf_mse}')
print(f'XGBoost R2 Score: {r2_score(test_y, xgb_preds)}')
```

```
XGBoost MSE: 40.85358471679688
XGBoost R2 Score: 0.9352314011153188
```

3. LightGBM Regressor

```python
# 3. LightGBM Regressor
lgb_model = LGBMRegressor(n_estimators=100, learning_rate=0.1, random_state=42)
lgb_model.fit(X_train, y_train)

lgb_preds = lgb_model.predict(X_val)
lgb_mse = mse(y_val, lgb_preds)
print(f'LightGBM MSE: {lgb_mse}')
print(f'LightGBM R2 Score: {r2_score(y_val, lgb_preds)}')
```

```
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.000852 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 4259
[LightGBM] [Info] Number of data points in the train set: 6744, number of used features: 31
[LightGBM] [Info] Start training from score 38.827106
LightGBM MSE: 98.93150872442077
LightGBM R2 Score: 0.964328014109175
```

```python
# Test 데이터로 성능을 평가해보세요.
lgb_preds = lgb_model.predict(test_x_scaled)
lgb_mse = mse(test_y, lgb_preds)
print(f'LightGBM MSE: {lgb_mse}')
print(f'LightGBM R2 Score: {r2_score(test_y, lgb_preds)}')
```

```
LightGBM MSE: 39.01515159205324
LightGBM R2 Score: 0.9299677824511514
```

4. CatBoost Regressor

```python
# 4. CatBoost Regressor
cat_model = CatBoostRegressor(iterations=100, learning_rate=0.1, depth=6, random_seed=42, verbose=0)
cat_model.fit(X_train, y_train)  # 범주형 컬럼 리스트 추가

# 예측 및 성능 평가
cat_preds = cat_model.predict(X_val)
cat_mse = mse(y_val, cat_preds)
print(f'CatBoost MSE: {cat_mse}')
print(f'CatBoost R2 Score: {r2_score(y_val, cat_preds)}')
```

```
CatBoost MSE: 61.198919813659415
CatBoost R2 Score: 0.9779333497257411
```

```python
# Test 데이터로 성능을 평가해보세요.
cat_preds = cat_model.predict(test_x_scaled)
cat_mse = mse(test_y, cat_preds)
print(f'CatBoost MSE: {cat_mse}')
print(f'CatBoost R2 Score: {r2_score(test_y, cat_preds)}')
```

```
CatBoost MSE: 44.48738933416246
CatBoost R2 Score: 0.9201451128370093
```

1. 3번파일_모델링(성능 최적화)

	RF	XGB	Light	Cat
MSE	72.4	99.2	98.9	61.1
R2 score	0.97	0.96	0.96	0.97

1. 3번파일_모델링(성능 최적화)

하이퍼파라미터 튜닝

1. 3번파일_모델링(성능 최적화)

```python
# 하이퍼파라미터 공간 정의
space = {
    'n_estimators': hp.choice('n_estimators', range(50, 300)),
    'max_depth': hp.choice('max_depth', range(5, 30)),
    'min_samples_split': hp.choice('min_samples_split', range(2, 10)),
    'min_samples_leaf': hp.choice('min_samples_leaf', range(1, 5))
}
```

```python
# 목적 함수 정의
def objective(params):
    model = RandomForestRegressor(**params, random_state=42, n_jobs=-1)
    # 교차 검증을 통해 모델 성능 평가
    score = -cross_val_score(model, train_x, train_y, cv=5, scoring='neg_mean_squared_error').mean()
    return {'loss': score, 'status': STATUS_OK}
```

교차검증으로 평균 MSE를 계산

→ MSE는 작을수록 좋지만, Hyperopt는 "작은 loss"를 최소화하므로 부호를 반전시킴

1. 3번파일_모델링(성능 최적화)

```python
# 최적화 실행
trials = Trials()
best = fmin(fn=objective, space=space, algo=tpe.suggest, max_evals=100, trials=trials)

# 최적의 하이퍼파라미터 출력
print("Best hyperparameters:", best)

# 최적의 하이퍼파라미터로 모델 학습
best_params = {
    'n_estimators': best['n_estimators'],
    'max_depth': best['max_depth'],
    'min_samples_split': best['min_samples_split'],
    'min_samples_leaf': best['min_samples_leaf']
}

rf_model = RandomForestRegressor(**best_params, random_state=42, n_jobs=-1)
rf_model.fit(train_x, train_y)

# Test 데이터로 성능을 평가해보세요.
predictions_rf = rf_model.predict(test_x)
mse_value_rf = mse(test_y, predictions_rf)
r2_value_rf = r2_score(test_y, predictions_rf)

print(f'Random Forest Model with Best Hyperparameters MSE: {mse_value_rf}')
print(f'Random Forest Model with Best Hyperparameters R2 Score: {r2_value_rf}')

# # 학습한 모델을 파일로 저장해보세요.
# joblib.dump(rf_model, 'rf_model.pkl')
```

```
100%|██████████| 100/100 [15:44<00:00,  9.45s/trial, best loss: 318.6895518304703]
Best hyperparameters: {'max_depth': 12, 'min_samples_leaf': 3, 'min_samples_split': 7, 'n_estimators': 19}
Random Forest Model with Best Hyperparameters MSE: 38.07375729390235
Random Forest Model with Best Hyperparameters R2 Score: 0.9316575857095547
```

fmin 함수

-TPE 알고리즘

-최대 100회 시도

-최적의 하이퍼파라미터

1. 3번파일_모델링(성능 최적화)

튜닝 전	→	튜닝 후
72.3	MSE	38.0
0.97	R2 score	0.93

1. 3번파일_모델링(성능 최적화)

AutoML

- 모델 비교
- 하이퍼파라미터 튜닝
- 앙상블(Stacking)
- 시각화 자동화

1. 3번파일_모델링(성능 최적화)

- 입력 데이터 정규화
- PyCaret-> 환경 세팅
- 모델 비교

```python
class FineDustPredictor:
    def __init__(self, train_x_path, train_y_path, test_x_path, test_y_path):
        self.train_x = pd.read_csv(train_x_path)
        self.train_y = pd.read_csv(train_y_path)
        self.test_x = pd.read_csv(test_x_path)
        self.test_y = pd.read_csv(test_y_path)
        # self.scaler = StandardScaler()
        self.scaler = MinMaxScaler()
        self.tuned_models = []
        self.results = {}
        self.pre_tuning_results = {}

    def normalize_data(self):
        """Normalize the training and testing datasets."""
        self.train_x = pd.DataFrame(self.scaler.fit_transform(self.train_x), columns=self.train_x.columns)
        self.test_x = pd.DataFrame(self.scaler.transform(self.test_x), columns=self.test_x.columns)

    def setup_autoML(self):
        """Set up PyCaret for AutoML and compare models."""
        self.data = pd.concat([self.train_x, self.train_y], axis=1)
        setup(data=self.data, target=self.train_y.columns[0], session_id=42, use_gpu=True)

    def compare_models(self):
        """Compare models based on R2 score and return top 5 models."""
        self.best_models = compare_models(sort='R2', n_select=5)
        return self.best_models
```

1. 3번파일_모델링(성능 최적화)

- 하이퍼파라미터 튜닝

```python
def tune_and_save_models(self):
    """Tune hyperparameters for each of the top models and save them."""
    for model in self.best_models:
        # Save performance metrics before tuning
        self.pre_tuning_results[str(model).split('(')[0]] = {}

        # Evaluate model before tuning
        self.evaluate_models([model], self.pre_tuning_results)

        tuned_model = tune_model(model, optimize='R2')
        self.tuned_models.append(tuned_model)

        # Save original model
        model_name = str(model).split('(')[0]  # Extract model name
        with open(f'./model_{model_name}.pkl', 'wb') as f:
            pickle.dump(model, f)

        # Save tuned model
        with open(f'./tuned_model_{model_name}.pkl', 'wb') as f:
            pickle.dump(tuned_model, f)

        # Evaluate tuned model
        self.evaluate_models([tuned_model], self.results)

    # Call the stacking method after tuning models
    self.stack_models()
```

1. 3번파일_모델링(성능 최적화)

- 스태킹 앙상블

```python
def stack_models(self):
    """Stack tuned models and evaluate performance."""
    # Prepare base model predictions
    X_meta = []
    y_meta = self.train_y['PM10_1']

    for model in self.tuned_models:
        train_predictions = predict_model(model, data=self.train_x)['prediction_label']
        X_meta.append(train_predictions)

    X_meta = np.array(X_meta).T

    # Split into training and validation sets for stacking
    X_train_meta, X_val_meta, y_train_meta, y_val_meta = train_test_split(X_meta, y_meta, test_size=0.2, random_state=42)

    # Define the stacking model
    stacker = StackingRegressor(
        estimators=[(str(model).split('(')[0], model) for model in self.tuned_models],
        final_estimator=LinearRegression(),
        cv=5
    )

    # Train the stacking model
    stacker.fit(X_train_meta, y_train_meta)

    # Make predictions
    predictions = stacker.predict(X_val_meta)

    # Evaluate the stacking model
    r2 = r2_score(y_val_meta, predictions)
    mse = mean_squared_error(y_val_meta, predictions)

    # Store the stacking results
    self.results['Stacked Model'] = {'R2 Score': r2, 'MSE': mse}
    print(f'Stacked Model - R2 Score: {r2:.4f}, MSE: {mse:.4f}')
```

2. 자율 프로젝트 주제 선정

2. 자율 프로젝트

- 전세사기 위험 예측 지도
- 혼잡도 기반 힐링 산책 코스 추천
- 주차난/불법주차 발생 예측 지도

등등....

2. 자율 프로젝트

자율 프로젝트 주제:

날씨 기반 코디 추천 시스템

2. 자율 프로젝트

주제 선정 배경:

- 매일 기온에 맞춰 “오늘 뭐 입지" 고민 하는 사람 많음

-> N년전과 비슷한 날씨를 찾아, 유사 조건의 코디 추천

2. 자율 프로젝트

목표 & 주요 기능

목표: “날씨에 맞는 스타일, 어떻게 입을까?”

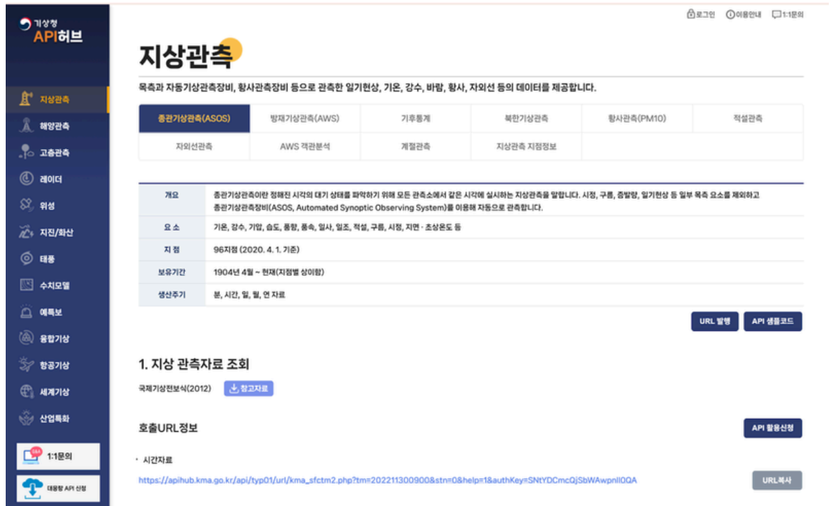
주요 기능:

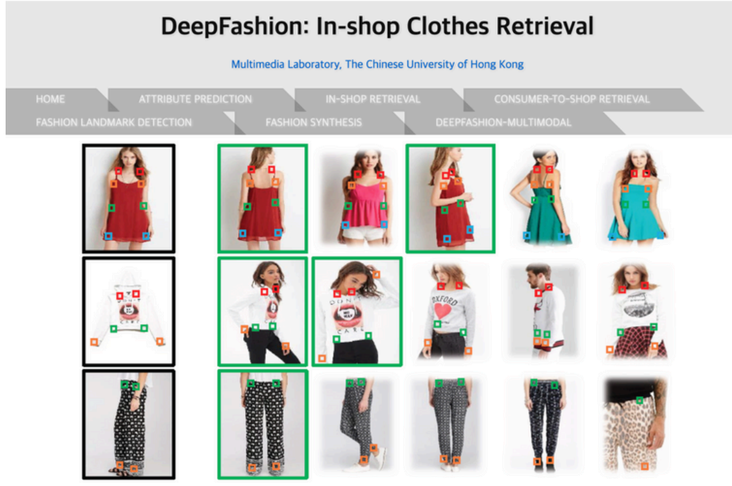
- 날짜 입력 → 기온 자동 파악
- 3년 날씨 데이터에서 유사한 날 찾기
- 기온에 맞는 코디 이미지 추천
- 추천 이미지 + 설명 출력

2. 자율 프로젝트

사용 데이터

구분	데이터	활용 목적
n개 년 치 전국 기온	기상청 기후 데이터	날짜 기반
코디 이미지 데이터셋	Kaggle, DeepFashion 등	착장 이미지 및 카테고리 태깅
매핑용 라벨 데이터	기온과 이미지 간 조건 정리 CSV	유사 날씨 - 코디 매칭 기반 구축





2. 자율 프로젝트

구현 계획:

Step1: 이미지 임베딩

- CNN 기반 의류 이미지 벡터화
- 각 이미지 간 유사도 계산이 가능한 상태로 변환

Step2: 유사 날씨 기반 추천

- 입력: 날짜 → 기온 파악
- 처리: 유사한 기온 조건을 가진 코디 이미지 탐색
- 출력: 코디 이미지 + 간단한 메시지

감사합니다.

AI+X 중급 2차 프로젝트

중간점검발표 -2